管理 Astra Trident

Astra Trident

NetApp
June 01, 2022

This PDF was generated from https://docs.netapp.com/zh-cn/trident-2110/trident-managing-k8s/upgrade-trident.html on June 01, 2022. Always check docs.netapp.com for the latest.

目录

管理 Astra Trident . . . 1
升级 Astra Trident . . . 1
使用操作员升级 . . . 2
使用 tridentctl 进行升级 . . . 10
卸载 Astra Trident . . . 13
降级 Astra Trident . . . 15

管理 Astra Trident

升级 Astra Trident

Astra Trident 遵循季度发布节奏，每个日历年提供四个主要版本。每个新版本都是在先前版本的基础上构建的，可提供新功能和性能增强以及错误修复和改进功能。我们建议您每年至少升级一次，以利用 Astra Trident 中的新功能。

> 要升级到之前的五个版本，您需要执行多步升级。

确定要升级到的版本

- 您可以从 `YY-1.mm` 版本和任何介于两者之间的版本升级到 `YY.mm` 版本。例如，您可以从 19.07 及更高版本直接升级到 20.07 （包括 DOT 版本，例如 19.07.1 ）。
- 如果您使用的是早期版本，则应执行多步骤升级。这要求您首先升级到适合您的四个版本窗口的最新版本。例如，如果您运行的是 18.07 ，并希望升级到 20.07 版本，请按照以下所述执行多步骤升级过程：
  - 首次从 18.07 升级到 19.07 。有关升级的具体说明，请参见相应版本的文档。
  - 然后从 19.07 升级到 20.07 。

> 对于 19.04 及更早版本的所有升级，都需要将 Astra Trident 的元数据从其自身的 `etcd` 迁移到 CRD 对象。请务必查看此版本的文档，了解升级的工作原理。

> 升级时，请务必提供 `parameter.FSType` in `StorageClasses` used by Astra Trident 。您可以删除并重新创建 `StorageClasses` ，而不会中断已有卷。这是执行的一项 * 要求 * "安全上下文" SAN 卷。。 "输入示例：" 目录包含示例，例如 “storage-class-basic 。 yaml.temp” 和 “storage-class-brone-default.yaml”。有关详细信息，请参见 "已知问题"。

我应选择哪种升级路径?

您可以使用以下路径之一进行升级：

- 使用 Trident 运算符。
- 使用 `tridentctl` 。

> 从 Kubernetes 1.20 开始， CSI 卷快照现在是 GA 功能。在升级 Astra Trident 时，在执行升级之前，必须删除所有先前的 alpha snapshot CRS 和 CRD （卷快照类，卷快照和卷快照内容）。请参见 "本博客" 了解将 alpha 快照迁移到测试版 /GA 规范所涉及的步骤。

如果满足以下条件，则可以使用 Trident 操作符进行升级：

- 您正在运行 CSI Trident （ 19.07 及更高版本）。
- 您已安装基于 CRD 的 Trident 版本（ 19.07 及更高版本）。
- 您正在执行自定义安装（使用自定义 YAML ）。

> 如果您使用的是基于 `etcd` 的 Trident 版本（ 19.04 或更早版本），请勿使用运算符升级 Trident 。

如果您不想使用该运算符，或者您的自定义安装操作符不支持，则可以使用 `tridentctl` 进行升级。这是 Trident 19.04 及更早版本的首选升级方法。

对运算符进行了更改

Astra Trident 21.01 版为操作员引入了一些关键的架构变更，即：

- 操作符现在为 * 集群范围 * 。以前的 Trident 运算符实例（版本 20.04 到 20.10 ）为 * 命名空间范围 * 。集群范围内的运算符具有优势，原因如下：
  - 资源责任：操作员现在可以在集群级别管理与 Astra Trident 安装相关的资源。在安装 Astra Trident 过程中，操作员使用 `ownerReferences` 创建和维护多个资源。在集群范围的资源上维护 所有权参考 可能会在某些 Kubernetes 分销商（例如 OpenShift ）上引发错误。使用集群范围的运算符可缓解此问题。对于 Trident 资源的自动修复和修补，这是一项基本要求。
  - 卸载期间清理：要完全删除 Astra Trident ，需要删除所有关联的资源。命名空间范围的运算符可能会在删除集群范围的资源（例如 clusterRole ， ClusterRoleBinding-and PodSecurityPolicy ）时遇到问题，并导致清理不完整。集群范围的运算符可消除此问题描述。用户可以完全卸载 Astra Trident 并在需要时重新安装。
- `TridentProvider` 现已替换为 `TridentOrchestrator` 作为用于安装和管理 Astra Trident 的自定义资源。此外，还为 `TridentOrchestrator` 规范引入了一个新字段。用户可以使用 `spec.namespace` 字段指定必须安装 / 升级命名空间 Trident 。您可以查看一个示例 "此处"。

了解更多信息

- "使用 Trident 操作符进行升级"
- "使用 `tridentctl` 进行升级"

使用操作员升级

您可以使用操作员轻松升级现有的 Astra Trident 安装。

要使用运算符进行升级，应满足以下条件：

- 您应安装基于 CSI 的 Astra Trident 。要检查是否正在运行 CSI Trident ，请检查 Trident 命名空间中的 Pod 。如果它们遵循 `trident CSI - *` 命名模式，则表示您正在运行 CSI Trident 。
- 您应安装基于 CRD 的 Trident 。这表示 19.07 及更高版本的所有版本。如果您安装的是基于 CSI 的安装，则很可能是基于 CRD 的安装。
- 如果您已卸载 CSI Trident ，并且安装中的元数据仍然存在，则可以使用操作员进行升级。
- 在给定 Kubernetes 集群中的所有命名空间中，只应安装一个 Astra Trident 。
- 您应使用运行的 Kubernetes 集群 "1.17 及更高版本"。
- 如果存在字母快照 CRD ，则应使用 `tridentctl obliviate alpha-snapshot-crd` 将其删除。此操作将删除 alpha snapshot 规范的 CRD 。有关应删除 / 迁移的现有快照，请参见 "本博客"。

> 在 OpenShift 容器平台上使用操作符升级 Trident 时，应升级到 Trident 21.01.1 或更高版本。21.01.0 版发布的 Trident 运算符包含一个已知的问题描述，该 已在 21.01.1 中修复。有关详细信息，请参见 "GitHub 上的问题描述详细信息"。

升级集群范围的操作员安装

要从 * Trident 21.01 及更高版本 * 升级，请执行以下步骤。

步骤

1. 删除用于安装当前 Astra Trident 实例的 Trident 运算符。例如，如果要从 21.01 升级，请运行以下命令：

```
kubectl delete -f 21.01/trident-installer/deploy/bundle.yaml -n trident
```

2. （可选）如果要修改安装参数，请编辑安装 Trident 时创建的 `TridentOrchestrator` 对象。这些更改可能包括修改自定义 Trident 映像，从中提取容器映像的专用映像注册表，启用调试日志或指定映像提取密钥等。
3. 使用为新版本设置 Trident 操作符的 `bundle.yaml` 文件安装 Astra Trident 。运行以下命令：

```
kubectl create -f 21.10.0/trident-installer/deploy/bundle.yaml -n
trident
```

在此步骤中， 21.10.0 Trident 操作员将确定现有的 Astra Trident 安装并将其升级到与操作员相同的版本。

升级命名空间范围的操作员安装

要从使用命名空间范围的运算符（版本 20.07 至 20.10 ）安装的 Astra Trident 实例进行升级，需要遵循以下步骤：

步骤

1. 验证现有 Trident 安装的状态。要执行此操作，请检查 `TridentProvider` 的 * 状态 * 。状态应为 `installed` 。

```
$ kubectl describe tprov trident -n trident | grep Message: -A 3
Message:  Trident installed
Status:   Installed
Version:  v20.10.1
```

> 如果状态显示 `Updating` ，请确保先解决此问题，然后再继续。有关可能的状态值列表，请参见 "此处"。

2. 使用 Trident 安装程序随附的清单创建 `TridentOrchestrator` CRD 。

```
# Download the release required [21.01]
$ mkdir 21.07.1
$ cd 21.07.1
$ wget
https://github.com/NetApp/trident/releases/download/v21.07.1/trident-
installer-21.07.1.tar.gz
$ tar -xf trident-installer-21.07.1.tar.gz
$ cd trident-installer
$ kubectl create -f
deploy/crds/trident.netapp.io_tridentorchestrators_crd_post1.16.yaml
```

3. 使用其清单删除命名空间范围的运算符。要完成此步骤，您需要使用 `bundle.yaml` 文件来部署命名空间范围的运算符。您可以从获取 `bundle.yaml` "Trident 存储库"。请确保使用相应的分支。

> 在删除命名空间范围的运算符后以及安装集群范围的运算符之前，您应对 Trident 安装参数进行必要的更改（例如，更改 `tridentImage` ， `autosupportImage` ，私有映像存储库以及提供 `imagePullSecs` 的值）。有关可更新的参数的完整列表，请参见 "参数列表"。

```
#Ensure you are in the right directory
$ pwd
$ /root/20.10.1/trident-installer

#Delete the namespace-scoped operator
$ kubectl delete -f deploy/bundle.yaml
serviceaccount "trident-operator" deleted
clusterrole.rbac.authorization.k8s.io "trident-operator" deleted
clusterrolebinding.rbac.authorization.k8s.io "trident-operator" deleted
deployment.apps "trident-operator" deleted
podsecuritypolicy.policy "tridentoperatorpods" deleted

#Confirm the Trident operator was removed
$ kubectl get all -n trident
NAME                                READY   STATUS    RESTARTS   AGE
pod/trident-csi-68d979fb85-dsrmn    6/6     Running   12         99d
pod/trident-csi-8jfhf               2/2     Running   6          105d
pod/trident-csi-jtnjz               2/2     Running   6          105d
pod/trident-csi-lcxvh               2/2     Running   8          105d

NAME                  TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)
AGE
service/trident-csi   ClusterIP   10.108.174.125   <none>
34571/TCP,9220/TCP   105d

NAME                          DESIRED   CURRENT   READY   UP-TO-DATE
AVAILABLE   NODE SELECTOR                                       AGE
daemonset.apps/trident-csi   3         3         3       3            3
kubernetes.io/arch=amd64,kubernetes.io/os=linux   105d

NAME                          READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/trident-csi   1/1     1            1           105d

NAME                                     DESIRED   CURRENT   READY   AGE
replicaset.apps/trident-csi-68d979fb85   1         1         1
105d
```

此时，将删除 `trident` 操作符 `-xxxxxx-xxxxx` Pod 。

4. （可选）如果需要修改安装参数，请更新 `TridentProvider` 规范。这些更改可能包括修改私有映像注册表以从中提取容器映像，启用调试日志或指定映像提取密钥等。

```
$  kubectl patch tprov <trident-provisioner-name> -n <trident-namespace>
--type=merge -p '{"spec":{"debug":true}}'
```

5. 安装集群范围的运算符。

> 安装集群范围的运算符会启动将 `TridentProvider` 对象迁移到 `TridentOrchestrator` 对象的操作，删除 `TridentProvider` 对象和 `tridentProvisioner` CRD ，并将 Astra Trident 升级到所使用的集群范围运算符版本。在以下示例中， Trident 会升级到 21.07.1 。

> 使用集群范围的运算符升级 Astra Trident 会将 `tridentProvider` 迁移到同名的 `tridentOrchestrator` 对象。此操作由操作员自动处理。在升级过程中， Astra Trident 也会安装在与之前相同的命名空间中。

```
#Ensure you are in the correct directory
$ pwd
$ /root/21.07.1/trident-installer

#Install the cluster-scoped operator in the **same namespace**
$ kubectl create -f deploy/bundle.yaml
serviceaccount/trident-operator created
clusterrole.rbac.authorization.k8s.io/trident-operator created
clusterrolebinding.rbac.authorization.k8s.io/trident-operator created
deployment.apps/trident-operator created
podsecuritypolicy.policy/tridentoperatorpods created

#All tridentProvisioners will be removed, including the CRD itself
$ kubectl get tprov -n trident
Error from server (NotFound): Unable to list "trident.netapp.io/v1,
Resource=tridentprovisioners": the server could not find the requested
resource (get tridentprovisioners.trident.netapp.io)

#tridentProvisioners are replaced by tridentOrchestrator
$ kubectl get torc
NAME      AGE
trident   13s

#Examine Trident pods in the namespace
$ kubectl get pods -n trident
NAME                                READY   STATUS    RESTARTS   AGE
trident-csi-79df798bdc-m79dc        6/6     Running   0          1m41s
trident-csi-xrst8                   2/2     Running   0          1m41s
trident-operator-5574dbbc68-nthjv   1/1     Running   0          1m52s

#Confirm Trident has been updated to the desired version
$ kubectl describe torc trident | grep Message -A 3
Message:                Trident installed
Namespace:              trident
Status:                 Installed
Version:                v21.07.1
```

升级基于 **Helm** 的操作员安装

要升级基于 Helm 的操作员安装，请执行以下步骤。

步骤

1. 下载最新的 Astra Trident 版本。
2. 使用 `helm upgrade` 命令。请参见以下示例：

```
$ helm upgrade <name> trident-operator-21.07.1.tgz
```

其中 `trident` 操作符 `-21.071.tgz` 反映了要升级到的版本。

3. 运行 `Helm list` 以验证图表和应用程序版本均已升级。

> 要在升级期间传递配置数据，请使用 `-set` 。

例如，要更改默认值 `tridentDebug` ，请运行以下命令：

```
$ helm upgrade <name> trident-operator-21.07.1-custom.tgz --set
tridentDebug=true
```

如果您运行的是 `$tridentctl logs` ，则可以看到调试消息。

> 如果在初始安装期间设置了任何非默认选项，请确保这些选项包含在 upgrade 命令中，否则，这些值将重置为其默认值。

从非操作员安装升级

如果您的 CSI Trident 实例满足上述前提条件，则可以升级到最新版本的 Trident 操作符。

步骤

1. 下载最新的 Astra Trident 版本。

```
# Download the release required [21.07.1]
$ mkdir 21.07.1
$ cd 21.07.1
$ wget
https://github.com/NetApp/trident/releases/download/v21.07.1/trident-
installer-21.07.1.tar.gz
$ tar -xf trident-installer-21.07.1.tar.gz
$ cd trident-installer
```

2. 从清单中创建 `tridentorchator` CRD 。

```
$ kubectl create -f
deploy/crds/trident.netapp.io_tridentorchestrators_crd_post1.16.yaml
```

3. 部署操作员。

```
#Install the cluster-scoped operator in the **same namespace**
$ kubectl create -f deploy/bundle.yaml
serviceaccount/trident-operator created
clusterrole.rbac.authorization.k8s.io/trident-operator created
clusterrolebinding.rbac.authorization.k8s.io/trident-operator created
deployment.apps/trident-operator created
podsecuritypolicy.policy/tridentoperatorpods created

#Examine the pods in the Trident namespace
NAME                                READY   STATUS    RESTARTS   AGE
trident-csi-79df798bdc-m79dc        6/6     Running   0          150d
trident-csi-xrst8                   2/2     Running   0          150d
trident-operator-5574dbbc68-nthjv   1/1     Running   0          1m30s
```

4. 创建 `TridentOrchestrator` CR 以安装 Astra Trident 。

```
#Create a tridentOrchestrator to initate a Trident install
$ cat deploy/crds/tridentorchestrator_cr.yaml
apiVersion: trident.netapp.io/v1
kind: TridentOrchestrator
metadata:
  name: trident
spec:
  debug: true
  namespace: trident

$ kubectl create -f deploy/crds/tridentorchestrator_cr.yaml

#Examine the pods in the Trident namespace
NAME                                READY   STATUS    RESTARTS   AGE
trident-csi-79df798bdc-m79dc        6/6     Running   0          1m
trident-csi-xrst8                   2/2     Running   0          1m
trident-operator-5574dbbc68-nthjv   1/1     Running   0          5m41s

#Confirm Trident was upgraded to the desired version
$ kubectl describe torc trident | grep Message -A 3
Message:                Trident installed
Namespace:              trident
Status:                 Installed
Version:                v21.07.1
```

现有后端和 PVC 会自动可用。

使用 tridentctl 进行升级

您可以使用 `tridentctl` 轻松升级现有的 Astra Trident 安装。

升级到最新版本的 Astra Trident 时，请考虑以下事项：

- 从 Trident 20.01 开始，仅限测试版 "卷快照" 受支持。Kubernetes 管理员应注意安全地备份或将 alpha Snapshot 对象转换为测试版，以保留原有的 alpha Snapshot 。
- 测试版的卷快照引入了一组经过修改的 CRD 和一个快照控制器，这两个控制器都应在安装 Astra Trident 之前进行设置。

> "本博客" 讨论将 alpha 卷快照迁移到测试版格式所涉及的步骤。

卸载并重新安装 Astra Trident 可作为升级。卸载 Trident 时，不会删除 Astra Trident 部署所使用的永久性卷声明（ PVC ）和永久性卷（ PV ）。在 Astra Trident 脱机期间，已配置的 PV 仍可用，而 Astra Trident 将在恢复联机后为在此期间创建的任何 PVC 配置卷。

> 升级 Astra Trident 时，请勿中断升级过程。确保安装程序运行完毕。

升级后的后续步骤

要使用新 Trident 版本中提供的丰富功能（例如按需卷快照），您可以使用 `tridentctl upgrade` 命令升级卷。

如果存在旧卷，则应将其从 NFS/iSCSI 类型升级到 CSI 类型，以便能够使用 Astra Trident 中的一整套新功能。Trident 配置的原有 PV 支持传统功能集。

决定将卷升级到 CSI 类型时，请考虑以下事项：

- 您可能不需要升级所有卷。以前创建的卷将继续可访问并正常运行。
- 升级时， PV 可以作为部署 / 状态集的一部分挂载。不需要关闭部署 / 状态集。
- 升级时，您 * 无法 * 将 PV 连接到独立 POD 。在升级卷之前，您应关闭 POD 。
- 您只能升级绑定到 PVC 的卷。升级前，应删除和导入未绑定到 PVC 的卷。

卷升级示例

以下示例显示了如何执行卷升级。

1. 运行 `kubectl get pv` 以列出 PV 。

```
$ kubectl get pv
NAME                          CAPACITY     ACCESS MODES   RECLAIM POLICY
STATUS   CLAIM                   STORAGECLASS    REASON   AGE
default-pvc-1-a8475           1073741824   RWO            Delete
Bound    default/pvc-1           standard                 19h
default-pvc-2-a8486           1073741824   RWO            Delete
Bound    default/pvc-2           standard                 19h
default-pvc-3-a849e           1073741824   RWO            Delete
Bound    default/pvc-3           standard                 19h
default-pvc-4-a84de           1073741824   RWO            Delete
Bound    default/pvc-4           standard                 19h
trident                       2Gi          RWO            Retain
Bound    trident/trident                                   19h
```

目前， Trident 20.07 使用 `netapp.io/trident` 配置程序创建了四个 PV 。

2. 运行 `kubectl describe PV` 以获取 PV 的详细信息。

```
$ kubectl describe pv default-pvc-2-a8486

Name:            default-pvc-2-a8486
Labels:          <none>
Annotations:     pv.kubernetes.io/provisioned-by: netapp.io/trident
                 volume.beta.kubernetes.io/storage-class: standard
Finalizers:      [kubernetes.io/pv-protection]
StorageClass:    standard
Status:          Bound
Claim:           default/pvc-2
Reclaim Policy:  Delete
Access Modes:    RWO
VolumeMode:      Filesystem
Capacity:        1073741824
Node Affinity:   <none>
Message:
Source:
    Type:      NFS (an NFS mount that lasts the lifetime of a pod)
    Server:    10.xx.xx.xx
    Path:      /trid_1907_alpha_default_pvc_2_a8486
    ReadOnly:  false
```

PV 是使用 `netapp.io/trident` 配置程序创建的，其类型为 NFS 。要支持 Astra Trident 提供的所有新功能，应将此 PV 升级到 CSI 类型。

3. 运行 `tridenctl upgrade volume <name-o-trident -volume>` 命令将原有的 Astra Trident 卷升级到 CSI 规范。

```
$ ./tridentctl get volumes -n trident
+----------------------+---------+---------------+----------
+--------------------------------------+--------+---------+
|            NAME      |  SIZE   | STORAGE CLASS | PROTOCOL |
BACKEND UUID             | STATE  | MANAGED |
+----------------------+---------+---------------+----------
+--------------------------------------+--------+---------+
| default-pvc-2-a8486 | 1.0 GiB | standard      | file     | c5a6f6a4-
b052-423b-80d4-8fb491a14a22 | online | true    |
| default-pvc-3-a849e | 1.0 GiB | standard      | file     | c5a6f6a4-
b052-423b-80d4-8fb491a14a22 | online | true    |
| default-pvc-1-a8475 | 1.0 GiB | standard      | file     | c5a6f6a4-
b052-423b-80d4-8fb491a14a22 | online | true    |
| default-pvc-4-a84de | 1.0 GiB | standard      | file     | c5a6f6a4-
b052-423b-80d4-8fb491a14a22 | online | true    |
+----------------------+---------+---------------+----------
+--------------------------------------+--------+---------+

$ ./tridentctl upgrade volume default-pvc-2-a8486 -n trident
+----------------------+---------+---------------+----------
+--------------------------------------+--------+---------+
|            NAME      |  SIZE   | STORAGE CLASS | PROTOCOL |
BACKEND UUID             | STATE  | MANAGED |
+----------------------+---------+---------------+----------
+--------------------------------------+--------+---------+
| default-pvc-2-a8486 | 1.0 GiB | standard      | file     | c5a6f6a4-
b052-423b-80d4-8fb491a14a22 | online | true    |
+----------------------+---------+---------------+----------
+--------------------------------------+--------+---------+
```

4. 运行 `kubectl describe PV` 以验证此卷是否为 CSI 卷。

```
$ kubectl describe pv default-pvc-2-a8486
Name:            default-pvc-2-a8486
Labels:          <none>
Annotations:     pv.kubernetes.io/provisioned-by: csi.trident.netapp.io
                 volume.beta.kubernetes.io/storage-class: standard
Finalizers:      [kubernetes.io/pv-protection]
StorageClass:    standard
Status:          Bound
Claim:           default/pvc-2
Reclaim Policy:  Delete
Access Modes:    RWO
VolumeMode:      Filesystem
Capacity:        1073741824
Node Affinity:   <none>
Message:
Source:
    Type:              CSI (a Container Storage Interface (CSI) volume
source)
    Driver:            csi.trident.netapp.io
    VolumeHandle:      default-pvc-2-a8486
    ReadOnly:          false
    VolumeAttributes:      backendUUID=c5a6f6a4-b052-423b-80d4-
8fb491a14a22

internalName=trid_1907_alpha_default_pvc_2_a8486
                           name=default-pvc-2-a8486
                           protocol=file
Events:                <none>
```

通过这种方式，您可以将由 Astra Trident 创建的 NFS/iSCSI 类型的卷逐个升级到 CSI 类型。

卸载 Astra Trident

根据 Astra Trident 的安装方式，有多种卸载方法。

使用 Helm 卸载

如果您使用 Helm 安装了 Astra Trident ，则可以使用 `helm uninstall` 将其卸载。

```
#List the Helm release corresponding to the Astra Trident install.
$ helm ls -n trident
NAME          NAMESPACE       REVISION        UPDATED
STATUS          CHART                           APP VERSION
trident       trident         1               2021-04-20
00:26:42.417764794 +0000 UTC deployed        trident-operator-21.07.1
21.07.1

#Uninstall Helm release to remove Trident
$ helm uninstall trident -n trident
release "trident" uninstalled
```

使用 **Trident** 操作符卸载

如果您使用操作符安装了 Astra Trident ，则可以通过执行以下操作之一卸载它：

- 编辑 TridentOrchestrator 设置卸载标志： * 您可以编辑 TridentOrchestrator s并设置 Pec.uninstall=true 。编辑 `TridentOrchestrator` CR 并设置 `uninstall` 标志，如下所示：

  ```
  $  kubectl patch torc <trident-orchestrator-name> --type=merge -p
  '{"spec":{"uninstall":true}}'
  ```

  当 `uninstall` 标志设置为 `true` 时， Trident 操作员将卸载 Trident ，但不会删除 Trident Orchestrator 本身。如果要重新安装 Trident ，应清理 Trident Orchestrator 并创建新的 Trident 。

- 删除 TridentOrchestrator： * 删除用于部署 `Astra Trident` 的 Trident Orchestrator CR 后，您可以指示操作员卸载 Trident 。操作员将处理 `TridentOrchestrator` 的删除，并继续删除 Astra Trident 部署和子集，同时删除在安装过程中创建的 Trident Pod 。要完全删除 Astra Trident （包括其创建的 CRD ）并有效地擦除板，您可以编辑 `TridentOrchestrator` 以传递 `wipeout` 选项。请参见以下示例：

```
$ kubectl patch torc <trident-orchestrator-name> --type=merge -p
'{"spec":{"wipeout":["crds"],"uninstall":true}}'
```

这将完全卸载 Astra Trident 并清除与后端及其管理的卷相关的所有元数据。后续安装将视为全新安装。

> 只有在执行完全卸载时，才应考虑擦除 CRD 。此操作无法撤消。* 除非您希望重新启动并创建全新的 Astra Trident 安装，否则请勿擦除 CRD 。

使用 **tridentctl** 卸载

在 `tridentctl` 中运行 `uninstall` 命令，如下所示删除除 CRD 和相关对象之外与 Astra Trident 关联的所有资源，从而可以轻松地重新运行安装程序以更新到最新版本。

```
./tridentctl uninstall -n <namespace>
```

要完全删除 Astra Trident ，您应删除由 Astra Trident 创建的 CRD 的最终结果并删除这些 CRD 。

降级 Astra Trident

了解降级到早期版本的 Astra Trident 所涉及的步骤。

您可能会考虑降级的各种原因，例如：

- 应急规划
- 立即修复因升级而发现的错误
- 依赖关系问题，升级失败和不完整

何时降级

迁移到使用 CRD 的 Astra Trident 版本时，应考虑降级。由于 Astra Trident 现在使用 CRD 来保持状态，因此创建的所有存储实体（后端，存储类， PV 和卷快照）都已关联 CRD 对象，而不是写入到 `trident` PV （由早期安装的 Astra Trident 版本使用）中的数据。新创建的 PV ，后端和存储类均作为 CRD 对象进行维护。如果需要降级，则只能对使用 CRD （ 19.07 及更高版本）运行的 Astra Trident 版本尝试执行此操作。这是为了确保在降级后，当前 Astra Trident 版本上执行的所有操作均可见。

何时不降级

您不应降级到使用 `etcd` 保持状态（ 19.04 及更早版本）的 Trident 版本。降级后，使用当前 Astra Trident 版本执行的所有操作都不会反映出来。在回滚到早期版本时，新创建的 PV 不可用。在迁移回早期版本时，对后端，PV ，存储类和卷快照（已创建 / 更新 / 删除）等对象所做的更改对 Astra Trident 不可见。恢复到早期版本不会中断对已使用早期版本创建的 PV 的访问，除非已对其进行升级。

使用操作员安装 Astra Trident 时的降级过程

对于使用 Trident 操作员完成的安装，降级过程有所不同，不需要使用 `tridentctl` 。

对于使用 Trident 操作符完成的安装， Astra Trident 可以降级为以下任一项：

- 使用命名空间范围的运算符（ 20.07 - 20.10 ）安装的版本。
- 使用集群范围运算符（ 21.01 及更高版本）安装的版本。

降级为集群范围的运算符

要将 Astra Trident 降级为使用集群范围运算符的版本，请执行以下步骤。

步骤

1. "卸载 Astra Trident"。* 除非要完全删除现有安装，否则请勿删除这些 CRD
2. 删除集群范围的运算符。为此，您需要使用清单来部署操作员。您可以从获取 "Trident GitHub 报告"。确保切换到所需的分支。

3. 通过安装所需版本的 Astra Trident 继续降级。按照所需版本的文档进行操作。

降级到命名空间范围的运算符

本节总结了降级到 20.07 到 20.10 范围内的 Astra Trident 版本所涉及的步骤，该版本将使用命名空间范围的运算符进行安装。

步骤

1. "卸载 Astra Trident"。除非要完全删除现有安装，否则请勿删除 `CRD` 。确保已删除 tridentOrchestrator 。

```
#Check to see if there are any tridentorchestrators present
$ kubectl get torc
NAME        AGE
trident     20h

#Looks like there is a tridentorchestrator that needs deleting
$ kubectl delete torc trident
tridentorchestrator.trident.netapp.io "trident" deleted
```

2. 删除集群范围的运算符。为此，您需要使用清单来部署操作员。您可以从此处获取 "Trident GitHub 报告"。确保切换到所需的分支。
3. 删除 `tridentorchator` CRD 。

```
#Check to see if ``tridentorchestrators.trident.netapp.io`` CRD is
present and delete it.
$ kubectl get crd tridentorchestrators.trident.netapp.io
NAME                                      CREATED AT
tridentorchestrators.trident.netapp.io   2021-01-21T21:11:37Z
$ kubectl delete crd tridentorchestrators.trident.netapp.io
customresourcedefinition.apiextensions.k8s.io
"tridentorchestrators.trident.netapp.io" deleted
```

   已卸载 Astra Trident 。

4. 通过安装所需版本继续降级。按照所需版本的文档进行操作。

使用 **Helm** 降级

要降级，请使用 `helm rollback` 命令。请参见以下示例：

```
$ helm rollback trident [revision #]
```

使用 tridentctl 安装 Astra Trident 时的降级过程

如果您使用 `tridentctl` 安装了 Astra Trident ，则降级过程涉及以下步骤。此顺序将指导您完成从 Astra Trident 21.07 迁移到 20.07 的降级过程。

> 在开始降级之前，您应创建 Kubernetes 集群的 `etcd` 的快照。这样，您就可以备份 Astra Trident 的 CRD 的当前状态了。

步骤

1. 确保使用 `tridentctl` 安装 Trident 。如果您不确定如何安装 Astra Trident ，请运行以下简单测试：
   a. 列出 Trident 命名空间中的 Pod 。
   b. 确定集群中运行的 Astra Trident 的版本。您可以使用 `tridentctl` 或查看 Trident Pod 中使用的图像。
   c. 如果您 * 未看到 * a `tridentOrchestrator` ，（或） a `tridentprovisioner` ，（或）名为 `trident — operator-xxxxxxxxxx-xxxxx` 的 Pod ，则已安装 Astra Trident * 和 `tridentctl` 。
2. 使用现有的 `tridentctl` 二进制文件卸载 Astra Trident 。在这种情况下，您将使用 21.07 二进制文件卸载。

```
$ tridentctl version -n trident
+----------------+----------------+
| SERVER VERSION | CLIENT VERSION |
+----------------+----------------+
| 21.07.0        | 21.07.0        |
+----------------+----------------+

$ tridentctl uninstall -n trident
INFO Deleted Trident deployment.
INFO Deleted Trident daemonset.
INFO Deleted Trident service.
INFO Deleted Trident secret.
INFO Deleted cluster role binding.
INFO Deleted cluster role.
INFO Deleted service account.
INFO Deleted pod security policy.
podSecurityPolicy=tridentpods
INFO The uninstaller did not delete Trident's namespace in case it is
going to be reused.
INFO Trident uninstallation succeeded.
```

3. 完成此操作后，获取所需版本的 Trident 二进制文件（在此示例中为 20.07 ），并使用它安装 Astra Trident 。您可以为生成自定义 YAML "自定义安装" 如果需要，

```
$ cd 20.07/trident-installer/
$ ./tridentctl install -n trident-ns
INFO Created installer service account.
serviceaccount=trident-installer
INFO Created installer cluster role.               clusterrole=trident-
installer
INFO Created installer cluster role binding.
clusterrolebinding=trident-installer
INFO Created installer configmap.                  configmap=trident-
installer
...
...
INFO Deleted installer cluster role binding.
INFO Deleted installer cluster role.
INFO Deleted installer service account.
```

降级过程已完成。

版权信息

版权所有©2022 NetApp、Inc.。保留所有权利。Printed in the U.S.版权所涵盖的本文档的任何部分不得以任何形式或任何手段复制、包括影印、录制、 磁带或存储在电子检索系统中—未经版权所有者事先书面许可。

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

本软件由NetApp按"原样"提供、不含任何明示或默示担保、包括但不限于适销性和特定用途适用性的默示担保、特此声明不承担任何责任。IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice.NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp.The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S.patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

商标信息

NetApp、NetApp标识和中列出的标记 http://www.netapp.com/TM 是NetApp、Inc.的商标。其他公司和产品名称可能是其各自所有者的商标。